

1. Usar abstracción

La abstracción consiste en **esconder los detalles internos** y quedarte solo con lo esencial.
Sirve para:

- Simplificar problemas complejos dividiéndolos en partes más manejables.
- Reducir errores al mantener menos detalles expuestos.
- Facilitar cambios futuros porque los módulos internos pueden modificarse sin romper lo externo.

2. Aplicar modularidad

La modularidad es **dividir el software en componentes pequeños, independientes y coherentes**.

Un sistema modular tiene:

- **Alta cohesión:** cada módulo hace una tarea clara.
- **Bajo acoplamiento:** los módulos dependen poco entre sí.

Esto permite:

- Reutilizar código.
- Facilitar el testing.
- Mantener y mejorar el sistema sin romper todo.

3. Realizar pruebas (testing)

El testing asegura que el código:

- Funciona como debe.
- No rompe otros componentes al añadir cambios.
- Cumple con los requerimientos.

Tipos relevantes:

- **Unit tests:** prueban funciones individuales.
- **Integration tests:** prueban cómo interactúan distintos módulos.
- **Regression tests:** aseguran que algo que antes funcionaba siga funcionando después de cambios.

4. Documentar

La documentación ayuda a entender:

- Qué hace el código.
- Cómo usarlo.
- Cómo mantenerlo.

Incluye:

- Comentarios útiles.
- Docstrings claros (en Python, C, etc.).
- README, diagramas o ejemplos de uso.

Una buena documentación **reduce la dependencia de una sola persona** y acelera el trabajo en equipo.



Práctica: control de versiones y code reviews

Control de versiones

Con Git u otras herramientas aprendes a:

- Guardar cambios de forma progresiva.
- Organizar trabajo con branches.
- Colaborar sin pisar el código de otros.
- Volver atrás cuando algo falla.

Buenas prácticas:

- Commits **pequeños** y frecuentes.
- Mensajes de commit claros.
- No mezclar funcionalidades distintas en un solo commit.

Revisiones de código (code reviews)

Las revisiones de código sirven para:

- Mejorar la calidad del código.
- Detectar errores antes de fusionar.
- Mantener un estilo consistente entre todos.
- Aprender buenas prácticas del equipo.

Una buena revisión mira:

- Legibilidad
- Estilo
- Seguridad
- Performance (cuando aplica)
- Claridad de la lógica
- Posibles errores o casos borde